2026年4月7日

OshiruZine-AI

人工知能・機械学習・LLMを中心に、
技術トレンドから実用事例まで幅広く扱う技術系雑誌。

OshiruZine-AI

2026年4月7日 号

目次

1. Claude Code/Codexでのソフトウェア開発の最新ベストプラクティス　*スタンダード（約2000字）*
2. GPT-5・Gemini 2.5・Claude 4——2026年春のフロンティアモデル総点検　*特集（約4000字）*
3. AIエージェント時代のセキュリティ新常識——プロンプトインジェクションからツール権限管理まで　*スタンダード（約2000字）*
4. オンデバイスSLM入門——スマホとエッジで動く小規模言語モデルの選び方　*ショート（500〜1000字）*

Claude Code/Codexでのソフトウェア開発の最新ベストプラクティス

2026年に入り、AIコーディングアシスタントはもはや「便利なツール」ではなく「開発チームの一員」として位置づけられるようになった。とりわけ、AnthropicのClaude CodeとOpenAIのCodexは、ターミナルネイティブなエージェント型開発ツールとして急速に成熟し、日々のソフトウェア開発ワークフローを根本から変えつつある。本コラムでは、2026年4月時点での最新動向を踏まえ、これらのツールを実務で最大限に活用するためのベストプラクティスを整理する。

エージェント型コーディングの現在地

2025年後半から2026年にかけて、AIコーディングツールは「補完型」から「エージェント型」へと明確にシフトした。Claude Codeはターミナル上で動作し、ファイルの読み書き、テスト実行、Git操作までを自律的にこなす。一方、OpenAI Codexも2025年5月のリリース以降アップデートを重ね、クラウドサンドボックス内での並列タスク実行やCI連携機能を強化してきた。

両ツールに共通するのは、「人間が指示を出し、AIがコードベース全体を理解した上で実装・検証まで一気通貫で行う」というパラダイムだ。この前提を正しく理解しているかどうかで、開発生産性に数倍の差が生まれる。

CLAUDE.md／指示ファイルの設計が成否を分ける

Claude Codeを使いこなす上で最も重要なのは、プロジェクトルートに配置する`CLAUDE.md`ファイルの設計だ。これはAIエージェントに対する「プロジェクトの憲法」のような役割を果たす。

最新のベストプラクティスとしては以下が挙げられる。

- **アーキテクチャの概要を簡潔に記述する**：ディレクトリ構造、主要な依存関係、設計パターンを明示する。AIが不必要な探索に時間を費やすことを防ぐ。
- **コーディング規約を具体的に指定する**：「関数名はcamelCase」「エラーハンドリングはResult型を使う」など、曖昧さのないルールを列挙する。
- **禁止事項を明記する**：「本番のデータベースに直接接続しない」「既存のテストを削除しない」など、安全性に関わる制約を書く。
- **テスト実行コマンドを記載する**：`npm test`や`pytest`などの具体的なコマンドを書いておくことで、AIが変更後に自動でテストを回せる。

Codex側でも同様に、リポジトリ内の`AGENTS.md`やシステムプロンプト設定で同等の指示が可能だ。いずれのツールでも、**指示ファイルの品質がそのまま出力の品質に直結する**という点は覚えておきたい。

タスク分割と段階的実行の原則

大規模な機能実装をAIに一括で任せたくなる気持ちは理解できるが、2026年時点でも「一度に一つの明確なタスク」が原則だ。

効果的なアプローチは以下の通りである。

1. **計画フェーズ**：まずAIに実装計画を立てさせ、人間がレビューする。Claude Codeでは`--plan`モードや「まず計画だけ出して」という指示が有効だ。
2. **実装フェーズ**：計画を承認した上で、ステップごとに実装を進める。各ステップ後にdiffを確認する習慣をつける。
3. **検証フェーズ**：テスト実行と静的解析を必ず挟む。Claude Codeは変更後に自動でテストを走らせる設定が可能であり、これを活用しない手はない。

Codexの場合は、複数のタスクを並列でサンドボックス実行できる点が強みだ。たとえば「APIエンドポイントの追加」と「対応するフロントエンドコンポーネ

ントの作成」を同時に走らせ、最後にインテグレーションする戦略が取れる。

テスト駆動との相性の良さを活かす

AIエージェント型開発とTDD（テスト駆動開発）の相性は極めて良い。テストが存在することで、AIは「自分の変更が正しいか」を自律的に判断できるからだ。

実践的には、**人間がテストケースを書き、AIが実装する**というワークフローが安定した成果を出す。テストという形で要件を厳密に伝えられるため、自然言語の曖昧さに起因するミスが大幅に減る。既存テストのカバレッジが高いプロジェクトほど、AIエージェントの恩恵を受けやすいという相関も、複数の開発チームから報告されている。

セキュリティとコードレビューの新しい視点

AIが生成したコードには、人間とは異なるパターンの脆弱性が混入する可能性がある。2025年から2026年にかけて、AIコード特有のセキュリティリスクに関する研究も進んでおり、以下の対策が推奨されている。

- **権限を最小化する**：Claude Codeのパーミッション設定を適切に行い、不要なファイルシステムアクセスやネットワーク接続を制限する。Codexのサンドボックスもネットワーク無効がデフォルトであり、この設計思想は正しい。
- **AIの出力を盲信しない**：生成されたコードのdiffは必ず人間がレビューする。特に認証・認可、暗号化、外部入力の処理に関わる部分は重点的に確認する。
- **AI同士のクロスレビュー**：Claude Codeで生成したコードをCodexにレビューさせる（あるいはその逆）という手法も実用段階に入っている。異なるモデルの視点を組み合わせることで、単一モデルの盲点を補完できる。

チーム開発への統合

個人の生産性向上にとどまらず、チーム開発プロセスへの統合が2026年の重要テーマだ。GitHub ActionsやGitLab CIにCodexを組み込み、プルリクエストごとに自動レビューや改善提案を行う運用は、すでに多くの組織で標準化されつつある。Claude CodeもGitHub連携を強化しており、Issue起点での自動実装提案が現実のものとなっている。

ただし、AI生成コードの著作権やライセンスに関する法的整理は各国で進行中であり、エンタープライズ利用においてはガバナンスポリシーの策定が不可欠だ。

まとめ：人間の判断力がこれまで以上に問われる

Claude CodeとCodexは、2026年のソフトウェア開発において欠かせないツールとなった。しかし、これらを真に使いこなすには、明確な指示設計、段階的なタスク管理、テストによる品質保証、そしてセキュリティ意識という、従来のエンジニアリングの基本原則がむしろ強化される形で求められる。AIが高度になればなるほど、それを制御する人間の設計力と判断力の価値は増す。ツールの進化に追従しつつ、エンジニアとしての本質的な力を磨き続けたい。

GPT-5・Gemini 2.5・Claude 4——2026年春のフロンティアモデル総点検

2026年の春は、大規模言語モデル（LLM）の勢力図が一気に塗り替わる季節になった。OpenAIは満を持してGPT-5をリリースし、Googleは Gemini 2.5 Pro/Ultra の大幅アップデートを敢行、そしてAnthropicはClaude 4ファミリーを投入した。3社がほぼ同時期にフロンティアモデルを刷新するのは前例がな

く、エンジニアや研究者にとっては「どれを使えばいいのか」がかつてないほど悩ましい状況になっている。

本稿では、2026年前半時点での主要3モデルを**推論コスト・長文コンテキスト・マルチモーダル性能**の三軸で横断的に整理し、ユースケースごとの選定指針を示す。ベンチマークの数字だけでは見えない"実戦での手触り"も含めて、できるだけ正直にレポートしたい。

2026年春、何が変わったのか——アーキテクチャの世代交代

まず、各モデルのアーキテクチャ面での進化を押さえておこう。

GPT-5は、OpenAIが「Project Strawberry」の延長線上で開発したとされる推論時計算スケーリング（inference-time compute scaling）を本格的にアーキテクチャレベルで統合した最初の商用モデルだ。従来のo1/o3系列で培われた「考える時間を長く取ることで精度を上げる」アプローチが、GPT-5ではベースモデルそのものに組み込まれた。ユーザーはAPIパラメータで推論の"深さ"を3段階（lite / standard / deep）に切り替えられ、タスクの難易度に応じてコストと精度のトレードオフを動的に制御できる。Dense Transformer をベースにしつつも、内部的にはMixture-of-Experts（MoE）的なルーティング機構が推論段階で活性化するハイブリッド構成と見られており、パラメータ総数は非公開ながら、アクティブパラメータは前世代比で抑制されているとの分析がある。

Gemini 2.5は、Googleが2025年末から段階的にロールアウトしてきたモデルの集大成だ。特筆すべきは、ネイティブマルチモーダルアーキテクチャのさらなる深化である。テキスト・画像・音声・動画・コードを単一のトークン空間で扱う設計は Gemini 1.5 から引き継がれているが、2.5 Ultra では空間認識（3Dシーン理解）と時系列推論（動画中のイベントの因果関係把握）が飛躍的に向上した。また、Googleのインフラ優位性を活かし、TPU v6e（Trillium）上で最適化されたサービングにより、トークン単価を競合比で大幅に引き下げている点も

見逃せない。コンテキストウィンドウは最大2Mトークンを維持しつつ、実効的な"忠実度"が改善されたとGoogleは主張している。

Claude 4は、Anthropicが安全性と実用性の両立を最大のテーマに掲げて送り出したモデルだ。Claude 3.5 Sonnet の高いコストパフォーマンスで多くの開発者を獲得したAnthropicは、Claude 4で「エージェント用途への最適化」を前面に押し出した。具体的には、ツール使用（function calling）の信頼性、長時間にわたるマルチターン対話での文脈維持、そして自己修正能力が大幅に強化されている。アーキテクチャの詳細は公開されていないが、Constitutional AIの進化版である「Alignment by Debate」的な内部機構が推論パイプラインに統合されており、ハルシネーション率がフロンティアモデルの中で最低水準を達成したとされる。コンテキストウィンドウはClaude 4 Opus で1Mトークン、Claude 4 Sonnet で500Kトークンとなっている。

ベンチマーク比較——数字で見る三つ巴

2026年3月時点で公開されている主要ベンチマークの結果を整理する。なお、ベンチマークのスコアは各社の公式発表およびサードパーティの検証（LMSYS Chatbot Arena、HELM、独立研究者によるリーク評価）を総合したものであり、評価条件が完全に統一されていない点には留意されたい。

ベンチマーク	GPT-5 (deep)	Gemini 2.5 Ultra	Claude 4 Opus
MMLU-Pro	93.2	92.8	91.5
GPQA Diamond	78.4	76.1	77.9
HumanEval+	95.7	93.4	94.8
SWE-bench Verified	62.3	58.7	64.1
MathOdyssey	89.1	88.5	86.3

ベンチマーク	GPT-5 (deep)	Gemini 2.5 Ultra	Claude 4 Opus
MMMU (マルチモーダル)	74.6	79.2	72.8
Video-Bench	68.3	81.7	65.2
Chatbot Arena Elo	1387	1379	1374

いくつかのポイントを読み解こう。

コーディング・ソフトウェアエンジニアリングでは、Claude 4 Opus がSWE-bench Verified で頭一つ抜けている。これはエージェント的なタスク遂行能力の高さを反映しており、実際のリポジトリに対するバグ修正やリファクタリングにおいてClaude 4の"粘り強さ"が効いている。一方、HumanEvalのような短いコード生成ではGPT-5が最高スコアを出しており、タスクの粒度によって優劣が入れ替わる。

数学・推論では、GPT-5の推論時計算スケーリングが威力を発揮している。deepモードで解かせた場合、GPQAやMathOdysseyで他モデルを上回るが、liteモードではClaude 4 Sonnetと同程度まで下がる。つまり「GPT-5が最強」と一口に言えるものではなく、推論コストとセットで評価する必要がある。

マルチモーダルは Gemini 2.5 Ultra の独壇場だ。特に動画理解（Video-Bench）では13ポイント以上の差をつけており、長時間動画からの情報抽出や、複数フレームにまたがる因果推論においてアーキテクチャ上の優位性が明確に表れている。

推論コスト——"最強モデル"は最適モデルではない

実プロダクトへの組み込みを考えるエンジニアにとって、ベンチマークのスコア以上に重要なのが推論コストだ。2026年4月時点のAPI公式価格（1Mトークンあたり、入力/出力）を比較する。

モデル	入力 ($/1M tokens)	出力 ($/1M tokens)	備考
GPT-5 lite	$3.00	$12.00	推論浅め・低遅延
GPT-5 standard	$8.00	$30.00	デフォルト設定
GPT-5 deep	$15.00	$60.00	推論最大深度
Gemini 2.5 Pro	$1.25	$5.00	128Kコンテキストまで
Gemini 2.5 Ultra	$5.00	$20.00	2Mコンテキスト対応
Claude 4 Sonnet	$3.00	$15.00	500Kコンテキスト
Claude 4 Opus	$15.00	$75.00	1Mコンテキスト

一目瞭然だが、**コスト効率でGemini 2.5 Proが圧倒的に優位**だ。Googleはクラウドインフラの垂直統合（自社TPU＋自社データセンター）を活かして、競合の半額以下を実現している。Batch APIを使えばさらに50%割引になるため、大量のドキュメント処理やデータパイプラインにおいてはGemini一択に近い状況が生まれている。

一方、Claude 4 Opusの出力$75/1Mトークンは高価に見えるが、SWE-benchのスコアが示すように「一発で正確なコードを生成し、手戻りが少ない」ことでトータルコストが下がるケースも多い。特にエージェントループで何度もリトライが発生するタスクでは、精度の高さが直接的なコスト削減につながる。

GPT-5は推論深度の3段階切り替えが最大の差別化ポイントだ。日常的なチャットやシンプルな要約にはliteモード（$3/$12）を使い、複雑な分析や数学問題にだけdeepモード（$15/$60）を回す、というルーティングを自動化できれば、平

均コストを大幅に抑えられる。OpenAIも公式にこの「アダプティブルーティング」パターンを推奨しており、SDK側にもそのためのヘルパーが追加されている。

長文コンテキスト——"入る"と"使える"は別問題

コンテキストウィンドウの名目上のサイズは、Gemini 2.5 Ultra の2Mトークンが最大だ。しかし、コンテキストが長くなるほど重要になるのは**検索忠実度（Needle-in-a-Haystack精度）**と**コンテキスト利用効率**である。

2026年3月にスタンフォード大のグループが発表した独立評価によると、100Kトークン以下ではGPT-5、Claude 4 Opus、Gemini 2.5 Ultraの三者にほとんど差がない（Needle-in-a-Haystack精度はいずれも99%以上）。差が出始めるのは**500Kトークンを超えた領域**だ。

- **Gemini 2.5 Ultra**: 1.5Mトークンまで95%以上の検索忠実度を維持。2Mトークンでも90%を下回らない。長大な動画トランスクリプトや法律文書の全文検索用途では他に代替がない。
- **Claude 4 Opus**: 500Kトークンまで97%の忠実度。700K〜1Mトークンでは90%前後まで緩やかに低下。ただし、情報の"統合"（複数箇所に散在する情報を結びつけて回答する能力）ではGemini 2.5を上回るケースが報告されている。
- **GPT-5**: 公式コンテキストウィンドウは256Kトークン。他2モデルに比べて名目上は小さいが、256K以内での精度は極めて高い。OpenAIはRAG（Retrieval-Augmented Generation）との併用を前提としたアーキテクチャ設計を公言しており、「すべてをコンテキストに詰め込む」アプローチとは異なる哲学を取っている。

実務上の示唆は明確だ。**「全文を一気に投入して分析したい」ならGemini 2.5 Ultra、「構造化された長文から的確に推論したい」ならClaude 4 Opus、「RAG**

パイプラインと組み合わせて効率よく処理したい」ならGPT-5というすみ分けが自然に生まれる。

マルチモーダル性能——テキスト以外の"知覚"はどこまで来たか

2026年春のフロンティアモデルは、いずれもテキスト以外の入力を扱えるが、その深度には大きな差がある。

Gemini 2.5 Ultraは、画像・音声・動画のネイティブ処理において依然として業界最高水準だ。特に動画理解は突出しており、1時間超の動画をそのまま入力し、特定のシーンの時刻指定つき要約や、登場人物の行動の因果関係分析が可能になっている。Google Lensとの統合によるリアルタイム空間認識も強力で、AR/VR用途の開発者にとっては事実上の第一選択肢だ。

GPT-5は、DALL·E 4ベースの画像生成をネイティブに統合した初のGPTモデルとなった。入力側でも画像・音声の理解力は高く、MMMUスコア74.6はClaude 4を上回る。ただし、動画入力は「フレームサンプリング＋分析」方式にとどまっており、Geminiのようなストリーミング動画理解には至っていない。音声入力についてはWhisper v4ベースのリアルタイム処理が利用可能で、会議の文字起こし＋要約の品質は極めて高い。

Claude 4は、マルチモーダル対応を「慎重に拡張」するAnthropicの方針が表れている。画像理解は高精度で、特にドキュメントOCR（手書き文字や複雑なテーブルの読み取り）では最高水準のパフォーマンスを示す。一方、音声や動画のネイティブ入力は2026年4月時点でまだベータ段階であり、プロダクション利用には制約がある。Anthropicはこの領域で急がない姿勢を見せており、「マルチモーダルの安全性評価が十分に完了してから段階的に開放する」とのスタンスだ。

ユースケース別・モデル選定ガイド

ここまでの分析を踏まえ、代表的なユースケースごとに推奨モデルを整理する。

1. コード生成・バグ修正・エージェント開発 → Claude 4 Opus / Sonnet を第一候補に。SWE-benchのスコアが示す通り、実コードベースに対するエージェント的タスクでの信頼性が高い。コスト重視ならClaude 4 Sonnetが好バランス。

2. データパイプライン・大量ドキュメント処理 → Gemini 2.5 Pro 一択に近い。圧倒的な低コストと長文コンテキスト、Batch APIの割引を活かせば、月間数百万リクエスト規模でも現実的な予算に収まる。

3. 数学・科学的推論・複雑な分析 → GPT-5 (deep) が最高精度。ただしコストが跳ね上がるため、まずstandardモードで試し、精度が不足する場合にのみdeepに切り替えるアダプティブ運用が推奨される。

4. マルチモーダル（動画分析・空間認識） → Gemini 2.5 Ultra。動画理解ではダブルスコアに近い差があり、競合は追いついていない。

5. 社内チャットボット・カスタマーサポート → GPT-5 lite または **Claude 4 Sonnet**。低遅延とコスト効率のバランスに優れ、ハルシネーション対策も進んでいる。Anthropicのガードレールの堅さを重視するならClaude、エコシステム（GPTs・Assistants API）の充実度を重視するならGPT-5。

6. RAG構成のナレッジベース検索 → GPT-5 がRAG前提の設計思想で最も洗練されている。Embedding APIとの親和性が高く、OpenAIのベクトルストア統合もスムーズ。

2026年後半を見据えて——エンジニアが今やるべきこと

最後に、少し先を見た話をしておきたい。

2026年後半にはMeta Llama 4の大規模版（400B

AIエージェント時代のセキュリティ新常識——プロンプトインジェクションからツール権限管理まで

あなたが設計したAIエージェントが、深夜に勝手にSlackへ社内機密を投稿していた——。これは仮定の話ではない。2025年後半から2026年初頭にかけて、自律型AIエージェントが外部ツールを不正に操作されるインシデント報告が業界内で急増している。エージェントが「考えて、判断して、行動する」時代になった今、セキュリティの前提そのものを書き換える必要がある。本コラムでは、2026年3月に公開されたOWASP LLM Top 10の改訂版を軸に、エージェント設計者が今すぐ押さえるべき防御パターンを整理する。

エージェント時代に攻撃面が激変した理由

従来のLLMアプリケーションは、ユーザーの入力を受け取りテキストを返すだけの「一問一答」型が主流だった。しかし2025年以降、LangChain、CrewAI、AutoGen、OpenAI Agents SDKなどのフレームワークが成熟し、LLMが外部API・データベース・ファイルシステムを自律的に呼び出す「エージェンティックワークフロー」が本番環境に急速に浸透した。

この変化がもたらしたのは、攻撃面（アタックサーフェス）の質的転換だ。テキスト生成の誤りが「不正確な回答」で済んでいた時代とは異なり、エージェントの判断ミスや操作の乗っ取りは、課金APIの不正利用、データベースの改ざん、機密情報の外部送信といった実害に直結する。LLMは今や「実行権限を持つア

クター」であり、セキュリティモデルも人間のオペレーターと同等以上の厳格さで設計しなければならない。

OWASP LLM Top 10（2026改訂版）が示す重点領域

2026年3月に公開されたOWASP LLM Top 10の改訂版では、エージェント固有のリスクカテゴリが大幅に拡充された。特に注目すべき変更点を3つ挙げる。

1. プロンプトインジェクションの細分化。 従来は「直接」「間接」の二分類だったが、改訂版ではエージェント間通信を介した「マルチホップインジェクション」が独立項目として追加された。エージェントAが取得した外部データに埋め込まれた悪意ある指示が、エージェントBのツール呼び出しをハイジャックするシナリオだ。

2. 過剰権限（Excessive Agency）の格上げ。 改訂版ではリスクランクが上昇し、ツール権限の最小化が最重要防御策として明記された。「便利だからとりあえず全権限を付与する」設計が、いかに危険であるかが改めて強調されている。

3. ツール・プラグインのサプライチェーンリスク。 サードパーティ製MCPサーバーやツール定義の改ざんを経由した攻撃が新規カテゴリとして加わった。2025年末に報告されたMCPプロトコル経由のツール定義ポイズニングが直接の契機とされている。

実装で押さえるべき5つの防御パターン

これらのリスクを踏まえ、エージェント設計時に組み込むべき防御パターンを整理する。

パターン1：最小権限の原則をツール単位で徹底する

エージェントに渡すツールは「そのタスクの遂行に必要な最小限」に絞る。たとえばデータ参照タスクには読み取り専用のAPIキーだけを渡し、書き込み権限は別のスコープで管理する。OpenAI Agents SDKやLangGraphでは、ワークフロ

ーのステップごとにツールセットを切り替える設計が可能だ。面倒でもステップ単位の権限分離を徹底してほしい。

パターン2：Human-in-the-Loopゲートの設置

破壊的操作（データ削除、外部送信、課金発生など）の前には、必ず人間の承認ステップを挟む。完全自律が理想であっても、リスクの高いアクションには「確認ゲート」を設けることで、インジェクション攻撃が成功しても被害を食い止められる。閾値ベースで自動承認と手動承認を切り替えるハイブリッド方式が実用的だ。

パターン3：入出力のサニタイズと境界マーキング

外部から取得したデータ（Webページ、メール、ドキュメント）をコンテキストに注入する際は、システムプロンプトとユーザーデータの境界を明示的にマークし、データ部分に含まれる指示的テキストをLLMが命令として解釈しないよう制御する。XMLタグやデリミタによる区画化に加え、入力テキストに対するインジェクション検知モデル（Rebuff、Lakera Guardなど）を前段に配置する多層防御が推奨される。

パターン4：ツール呼び出しのログと監査

エージェントがどのツールを、どの引数で、何回呼び出したかを構造化ログとして記録し、異常検知パイプラインに流す。「通常1回のAPI呼び出しが100回連続している」「未定義の引数パターンが出現した」などの逸脱をリアルタイムで検出できる体制を構築する。OpenTelemetryベースのLLMオブザーバビリティツール（Langfuse、Arize Phoenixなど）が実用段階に入っている。

パターン5：サプライチェーンの検証

MCPサーバーやサードパーティツールを導入する際は、ツール定義ファイルのハッシュ検証、提供元のレピュテーション確認、そしてサンドボックス環境での事前テストを必ず実施する。npmやpipのパッケージと同様、ツール定義にも「信頼の連鎖」が求められる時代だ。

セキュリティは「後付け」では間に合わない

エージェント開発の現場では、まずユースケースを動かし、セキュリティは後から考えるというアプローチがいまだに多い。しかしエージェントのセキュリティは、アーキテクチャの根幹に関わる設計判断だ。権限モデル、承認フロー、ログ基盤を後から差し込もうとすると、ワークフロー全体の再設計を迫られることになる。

2026年はエージェントが「実験」から「本番運用」へ本格移行する年になる。その転換点において、セキュリティを設計の初日から組み込む「セキュリティ・バイ・デザイン」の原則が、信頼されるAIシステムを構築するための最低条件となる。まずは自分のエージェントが持つツール一覧を開き、「この権限は本当に必要か？」と問いかけることから始めてみてほしい。

オンデバイスSLM入門——スマホとエッジで動く小規模言語モデルの選び方

もはや「スマホでLLMが動く」はデモのネタではない。2026年に入り、3B〜12Bクラスの小規模言語モデル（SLM）がモバイル・エッジデバイスで"実用水準"に到達した。クラウドへのリクエストなしに、手元のデバイスでテキスト生成・要約・コード補完がリアルタイムに完結する時代がいよいよ本格化している。本コラムでは、代表的なモデル・量子化手法・デプロイツールをコンパクトに整理する。

注目すべき3モデル

Phi-4 mini（3.8B） — Microsoftが送り出したSLMの決定版。パラメータ数の割に推論・コード生成の精度が高く、Windows Copilot+ PCやSnapdragon X搭載デバイスへの組み込み事例が急増中だ。ONNX Runtime経由でNPUをフル活用できる点も強い。

Gemma 3（4B / 12B） — Googleが2025年にリリースしたオープンウェイトモデル。12Bでもマルチモーダル対応（画像＋テキスト）が可能で、Android端末上でのMediaPipe統合パスが公式に整備されている。Pixelシリーズとの親和性は随一。

Llama 4 Scout（17Bだが有効パラメータは軽量） — Metaが投入したMoE（Mixture of Experts）構成のモデル。全体のパラメータ数は大きいが、推論時にアクティブとなるパラメータが限定されるため、メモリフットプリントはSLMクラスに抑えられる。多言語性能に優れ、日本語タスクでも安定した品質を見せる。

量子化がカギを握る

オンデバイス推論のボトルネックはメモリ帯域だ。FP16のまま載せるのは非現実的で、**4bit量子化（Q4_K_M等）** が事実上の標準になっている。GPTQ・AWQに加え、最近はllama.cppが採用する**GGUF形式のimatrix量子化**が精度と圧縮率のバランスで支持を集めている。12Bモデルでも4bit化すれば約7GBに収まり、RAM 8GBクラスのスマートフォンで動作する。

デプロイツールの選択肢

ツール	特徴
llama.cpp / llama-server	C++実装の軽量推論エンジン。iOSアプリへの組み込み実績も豊富

ツール	特徴
MediaPipe LLM Inference	Google公式。Android＋Gemmaの黄金パス
ExecuTorch	Meta発のPyTorchネイティブなオンデバイス推論フレームワーク。Llama系の最適化が手厚い
ONNX Runtime Mobile	NPU/DSP対応が広く、WindowsからAndroidまでクロスプラットフォーム展開に強い

まとめ——選定の指針

「まず試す」ならGGUF + llama.cpp、Android中心ならGemma 3 + MediaPipe、Windows/Snapdragon環境ならPhi-4 + ONNX Runtime、多言語・MoEの先進性を取るならLlama 4 Scout + ExecuTorchが最短ルートだ。いずれも活発にアップデートが続いており、数カ月単位で性能が跳ね上がる領域でもある。クラウドレイテンシとAPI課金から解放されたローカルAIの世界に、いま飛び込む価値は十分にある。